

2016 Ulsan-Fault ML-picker comparison

A study guide: how the relocation pipeline runs, and what the four pickers show

Detection → association → absolute location → QC → HypoDD dt.ct/dt.cc Consistent P=S=0.2 threshold

This document has two parts. **Part 1** walks the pipeline stage by stage so you can run and audit it yourself, with the origin/location provenance made explicit (the place a subtle bug once lived). **Part 2** is the corrected four-picker comparison and what it means scientifically.

1 Part 1 — How the pipeline runs

The engine is the `korea-cluster-relocation` pipeline (in `15.PocketQuake`), originally built to re-locate the KMA catalog. We feed our *own* PhaseNet+/PyOcto catalog into it unchanged. The *only* variable across the four runs is the ML picker; every stage after picking is identical.

The five invariants (keep these and the pipeline stays correct)

1. **One HypoInverse solution, computed once, reused everywhere.** QC only *selects* a subset; it never re-locates. The subset relocation reuses the full-run `.sum/.arc`.
2. **Pick provenance is by event_idx, never by timestamp.** An event's picks live in `event_sac/<event_idx>/<event_idx>_picks.csv`. A time-window key can grab a neighbour's picks.
3. **Origins flow into dt.cc, not just into start locations.** `rereference` stamps SAC origins from a `.sum`; `xcorr` then *subtracts* them. A wrong `.sum` corrupts `dt.cc` values.
4. **members.txt row order defines the cuspid** (200000 + row) tying together `.sum`, `.arc`, `event.dat`, `dt.cc`. Never reorder without regenerating all.
5. **The catalog is fed in KST and round-trips to UTC.** Do not “fix” it to UTC (see box below).

1.1 The stages

#	Stage	In → Out	Origin/location source
0	Detection	continuous mSEED → picks/*.parquet	— (raw picks, prob $\geq$ 0.2, P=S)
1	Association (PyOcto)	picks → catalog_*_pyocto.csv + assign	PyOcto (rough). Gate 4/2/2, ≥ 1 P+S
2	SAC store	assoc → event_sac/<idx>/*.sac + <idx>_picks.csv	PyOcto (in headers); <idx>_picks.csv = true pick set
3	UF-box catalog	→ catalog_kma.csv (KST), members.txt	defines cuspid = 200000+row
4	Absolute loc (HypoInverse, FULL)	assoc-pick .arc → 1.HypoInv/kim2011/*.sum	FULL-run HypoInverse = authoritative
5	QC (filter only)	full .sum → members_qc.txt	<i>no re-location</i> (see criteria below)
6	Re-reference	.sum origins → restamp SAC nz*, a/t0	must read the full-run .sum
7	ph2dt	.arc → event.dat, dt.ct	same .arc as stage 4/6
8	Cross-corr dt.cc (GPU)	re-ref SACs → dt.cc_0.7_combined	subtracts the stage-6 origins
9	HypoDD	event.dat + dt.ct + dt.cc → hypoDD.reloc	relative reloc; ISTART=2, adaptive damping

Driver: `run_picker_reloc.py --picker <p> --through dtcc` chains stages 2–9.

1.2 The two quality gates

PyOcto association gate (stage 1) **HypoInverse QC** (stage 5, all must hold)

total picks	≥ 4	erh	< 5 km (horizontal error)
P picks	≥ 2	erz	< 5 km (depth error)
S picks	≥ 2	gap	< 270° (azimuthal gap)
P+S stations	≥ 1	num	> 5 (≥ 6 phases)
velocity	kim2011 1-D	rms	< 1.0 s (residual RMS)

The association gate is deliberately *permissive*; the real quality control is the HypoInverse QC. HypoDD uses kim2011, ISTART=2 (mainshock fixed), and adaptive LSQR damping (condition number kept ~ 40 –80). “dt.cc-resolved” = an event keeping ≥ 1 surviving cross-correlation link (**nccp+nccs** > 0) — the highest-precision subset.

Why the catalog is “catalog_kma.csv” and in KST (a common confusion)

We do **not** use the KMA catalog. The pipeline hard-codes `kst_offset_hours = 9` (`originUTC = catalogKST - 9h`). So `build_catalog_kma.py` converts our PyOcto *UTC* origins to KST (+9h) *only so the pipeline subtracts 9h again* and lands back on the true UTC origin — a deliberate **no-op round-trip** to satisfy the pipeline’s KST-input contract. `catalog_kma.csv` is a *format label*, not a data source. Feeding UTC directly would place every event 9 h in the past.

1.3 The bug that was fixed (a worked example of the invariants)

The QC→dt.cc chain used to scaffold a *separate* QC cluster and **re-run HypoInverse + ph2dt** on it (violating invariant 1). That re-run re-staged picks by a second-resolution *timestamp* key (violating invariant 2), so on dense catalogs an adjacent event's raw picks overwrote the correct associated picks → wrong origins (up to ~1s off). Those wrong origins then poisoned **rereference** → **dt.cc values** (invariant 3) and **event.dat/dt.ct**. *Symptom that exposed it*: **event.sel** showed ERZ > 5 km events that had passed QC — because those errors came from the re-run, not the full run QC gated on. **Fix**: inject the full-run HypoInverse (**.sum/.arc**) subset to the QC members, then re-run rereference→ph2dt→xcorr→dtcc against correct origins. One HypoInverse solution, reused (now permanent in **run_picker_reloc.py::inject_full_hypoinverse**).

1.4 Reproducing the output yourself (run these in order)

All commands run from **Gyeongju_catalog/detection_test/**. The picker <p> is one of **original**, **stead**, **eqt** (PN+ is already done and is reused). <Y> is the year (2016 here). Stages A–C are *per month*; loop <mm> over 01..12. Stage 0 checks readiness; Stage D chains everything association-onward for one picker.

	Step	command
0	Readiness check (per year)	<code>python reloc_2016_uf/preflight_year.py --year <Y></code>
A	Station cache (per month)	<code>python lib/build_stations.py --month <Y>--<mm></code>
B	Detection (per picker, per month) (PN+: <code>run_pnplus_month.py</code>)	<code>python lib/run_seisbench_picker.py --model <p> --month <Y>--<mm></code>
C	Association (per picker, per month)	<code>python lib/associate_daily.py --picker <p> --month <Y>--<mm></code>
D	Relocation chain (once per picker)	<code>python reloc_2016_uf/run_picker_reloc.py --picker <p> --year <Y> --through dtcc</code>
E	Comparison notebooks	<code>jupyter nbconvert --execute --inplace reloc_2016_uf/10.Picker_comparison_2016.ipynb</code>

What stage D does internally (stages 2–9 of the table above): builds the event-idx SAC store → **catalog_kma/members** → scaffold + HypoInverse (full) → **uf_cluster QC** → *inject the full-run HypoInverse into the QC subset* (the fix) → rereference → ph2dt → GPU xcorr (**interp_hz=1000**, run in the **pq-gpu** conda env) → adaptive kim2011/ISTART=2 HypoDD. Stop earlier with **--through hypoinverse** to validate the chain up to QC (fast).

Practical notes

- Detection is CPU/IO-bound (GPU $\sim 27\%$) — run the three pickers’ months concurrently ($\sim 3\times$).
- The dt.cc xcorr is the long pole: ~ 6 h each for the dense pickers (PN+ 177k pairs, original 164k); STEAD/EQT are minutes. It is checkpointable (XCORR_RESUME=1).
- Everything is idempotent / safe to re-run; the QC injection backs up any prior HypoInverse to 1.HypoInv/kim2011.rerun_backup.

1.5 Does this inherit to every year 2010–2025 without adjustment?

Yes — the whole chain is `--year` general, and robust to network/rate change by design. Every layer is year-parameterized; the ones below marked “auto” need *no* per-year adjustment.

Layer	Year?	Network?	Rate?	Ad hoc?	How
Station cache (<code>build_stations.py</code>)	<code>--month</code>	auto	—	no	epochs/coords/bands read from StationXML; coverage = days on disk $\Rightarrow$ picks up whatever is deployed
Detection (<code>run_seisbench_picker.py</code>)	<code>--month</code>	auto	auto	no	anti-alias decimate per station; runs 2014/2016/2021 already
Association (<code>associate_daily.py</code>)	<code>--month</code>	auto	—	no	daily-chunked PyOcto; same fixed 4/2/2 gate every epoch
SAC store (<code>event_sac_export</code>)	—	auto	native	no	SAC_TARGET_HZ=None keeps native rate; xcorr interpolates at correlation time
Orchestration (<code>run_picker_reloc.py</code>)	<code>--year</code>	auto	auto	no	all year-dependent names via <code>year_paths.py</code> ; <code>--year 2016</code> = the exact existing paths

Robustness to network/rate change is built in, no tuning. Network change (KS/KG only in 2010 $\rightarrow$ +NS dense array 2017+ $\rightarrow$ +GJ temporary 2016) is handled automatically because stations are *discovered from metadata + on-disk coverage each month*, not listed. Sampling-rate change (100 / 200 / 1000 Hz across KS/KG/NS/GJ) is handled by the *native-rate* SAC store plus interpolate-at-correlation-time — this was the specific fix for the GJ mixed-rate loss. The velocity model (kim2011), QC thresholds, and the 4/2/2 gate are epoch-invariant by choice, so catalogs stay directly comparable across years. Archive coverage confirms the data exists and the network grows: 2010 has ~ 2300 KS/KG station-days, 2022 ~ 5900 .

Running any year (the orchestration is done)

`year_paths.py` centralizes every year-dependent name; `--year` defaults to 2016 and resolves to the exact existing 2016 paths/slugs, so 2016 work is byte-unchanged. Naming: `reloc_<Y>_uf[_<p>]/`, slug `uf_<Y>[_<p>]`, `stations_<Y>.csv`. To run a year:

```
python preflight_year.py --year <Y>          (which inputs exist / what to run)
python run_picker_reloc.py --picker <p> --year <Y> --through dtcc
```

What a new year still needs: only its Stage 0–1 *inputs* generated (detection + association for the 12 months — the heavy compute); the orchestration (Stages 2–9) then runs unchanged. *Real* per-year effects, none requiring code changes: network-gap months give smaller sets; sparse early years (2010–2012, KS/KG only) have larger azimuthal gaps so fewer events survive QC (the gate correctly reflects this rather than inventing precision); the `dt.cc` xcorr cost scales as $O(N^2)$ in events, so dense aftershock years are the expensive ones. These are the pipeline honestly reporting what each year’s network resolves.

2 Part 2 — The four-picker comparison (corrected)

Four ML pickers, one pipeline, consistent $P=S=0.2$ threshold. The question is not who *picks* most, but who yields the most well-located, cross-correlation-resolved events.

2.1 The funnel: picks → region events → UF-box → QC → dt.cc → dt.cc-resolved

picker	P picks	S picks	S/P	region ev.	UF-box	QC	dt.cc	cc-res.
PN+	533 394	638 063	1.20	28 168	3 867	596	512	255
PN-original	951 731	839 799	0.88	31 762	4 117	574	447	189
PN-STEAD	122 744	227 351	1.85	6 472	398	188	172	103
EQT-STEAD	87 396	95 638	1.09	6 540	424	160	144	92

How to read each column. *Picks* = raw detections (whole year, whole region). *Region ev.* = all PyOcto-associated events. *UF-box* = events inside the fault box (relocation members). *QC* = pass the HypoInverse quality gate. *dt.cc* = HypoDD-relocated with cross-corr. *cc-res.* = keep ≥ 1 surviving cc link — the headline, highest-precision number.

2.2 Fingerprints and headline

- **Pick quantity does not predict located quality.** PN-original picks the *most* (1.79 M) and associates the most region events (31 762), yet yields the *fewest* cc-resolved of the two PN models (189 vs PN+’s 255). Raw detection is an association liability when it is noisy.
- **S/P ratio is a picker fingerprint:** PN-original is P-heavy (0.88), PN-STEAD strongly S-heavy (1.85), PN+ balanced (1.20), EQT (1.09). The 4/2/2 gate rebalances this downstream.
- **A fixed 0.2 threshold is not equivalent across pickers.** The STEAD-retrained models (PN-STEAD, EQT) are far more conservative at 0.2 than NCEDC/EQNet (PN-original, PN+): $\sim 6\%$ of their region events fall in the UF box vs $\sim 13\%$ for the PN models.
- **Headline: PN+ delivers the most cross-correlation-resolved, well-located events (255)** despite not picking the most — the best balance of sensitivity and pick quality.

2.3 Why PN-original *dropped* after the fix (463→447; cc-res. 212→189)

This is the fix working, not a regression. The dt.cc value is $dt = (t_1 + \text{shift} - ot_1) - (t_2 - ot_2)$; the cross-correlation *coefficient* (which gates the 0.7 cut) is origin-independent, but the stored *value* carries the origins *ot*. PN-original, the noisiest picker, had the *largest* origin errors in the buggy run (**median 0.13 s; 121 events off by >0.5 s; 59 by >1 s**, vs PN+'s median 0.07 s). A *systematic per-event* offset is partly absorbable by relocating the event, so those links *survived* HypoDD's residual cuts (WDCC = 6, 4, 3, 2 → 0.5 s across the iteration sets) — keeping the event “cc-resolved” but at a wrong, offset-driven position. With the origins corrected, the truly inconsistent links show their real (large) residuals and are *correctly rejected*; events losing all cc links fall to ct-only, and marginal ones may not relocate. Hence a **smaller but genuinely better-located** set — confirmed by the relative error *improving* 0.72 → 0.50 m. PN+ barely moved (251 → 255) precisely because its origin errors were small.

One-sentence takeaway

The buggy origin offsets were coherent enough to be absorbed as fake relocations that passed HypoDD's residual cuts; correcting the origins exposes the truly-inconsistent links, which HypoDD then correctly rejects — so PN-original's count drops but its catalog becomes more trustworthy.

2.4 Corrected vs. buggy (all four)

picker	corrected / buggy			origin error (buggy)	
	dt.cc total	cc-resolved	rel-err (m)	median	#>0.5 s
PN+	512 / 511	255 / 251	0.42 / 0.42	0.07 s	few
PN-original	447 / 463	189 / 212	0.50 / 0.72	0.13 s	121
PN-STEAD	172 / 172	103 / 116	0.36 / 0.28	small	8
EQT-STEAD	144 / 146	92 / 101	0.42 / 0.28	small	6

All four have **zero air-quakes** (no depth < 0 or > 30 km) and healthy adaptive-damping convergence. The magnitude of each picker's change tracks its origin-error magnitude: PN-original (worst corruption) moves most; PN+ (least) barely moves.

Companion files: PIPELINE.md (invariants + provenance checks), notebooks 09/10/12 (comparison, funnel, record sections), run_picker_reloc.py + year_paths.py + preflight_year.py (year-general driver), fix_qc_rerun_bug.py (the repair driver).